Processor Hardware/Software Interfaces
EECS 113

Final Project

Hao Pham
52591771

06/13/2025

---

1 Objective

In this assignment, we will implement a Building Management System (BMS) using a Raspberry Pi. This system will integrate and automate multiple subsystems such as ambient temperature control, HVAC (heating, ventilation, and air conditioning), fire detection, motion-activated security lighting. To do so, we will code a Python program that interfaces with various sensors, actuators, and API systems to demonstrate real-time environmental monitoring and responsive control. For our project, we will use a PIR sensor to implement motion-activated lighting, DHT11 to collect ambient temperature, OpenWeatherMap API to collect the local humidity – all to implement the HVAC system. We will also use pushbuttons to implement security control and ambient temperature control, and LEDs to indicate different subsystems within the BMS. The status of every subsystem and real-time updates will be output onto an LCD and be logged onto a text file to track key system events with timestamps.

2 Procedure

2.1 Modules

```
import RPi.GPIO as GPIO
import time
from datetime import datetime
import LCD1602 as lcd1602
import Freenove_DHT as DHT
import requests, json
```

The modules we will be using for this project will include RPi.GPIO to interface with the GPIO pins of the raspberry pi, the time module to put the raspberry pi to sleep at certain parts of the

code, the datetime module to output the system's time in 24-hour format onto the LCD, the LCD1602 and Freenove_DHT modules provided by the Freenove Ultimate Starter Kit to interface with the LCD and DHT components, and the requests and json modules to collect the humidity from our OpenWeatherMap API.

2.2 Initialization

This function will serve to initialize all of the global variables that will be used and modified throughout the program, as well as set up the input/output and pins assignment for every hardware component used in our project.

```
# init everything
def init_everything():
    global lcd
    global pir_sensor
    global lights_led, lights_on, last_time_triggered
    global security_button, security_enabled
    global url, response, x, humidity
    global hvac_off, heat, ac, dht_pin, dht, temp_list, weather_index
    global inc_temp, dec_temp, desired_temp
    global ac_led, heater_led
    global fire_alarm
    global logfile
```

For the LCD, we will create an instance of the LCD class imported from the Freenove module so that we can write messages onto the LCD. For the pir_sensor variable, we will declare it as the GPIO pin 5 and set it up as input since it will be used to detect motion. For the lighting system, we will declare the lights LED as GPIO pin 16 and initialize it to output as low. We will also add a boolean variable called lights_on to be used in the PIR sensor function to check if the lights are already on or not, as well as a last_time_triggered variable that records that last time the PIR sensor detected motion or was triggered. This ensures that the lights LED stays on if the PIR sensor still detects motion within those 10 seconds.

```
# lcd
lcd = lcd1602.CharLCD1602()
lcd.init_lcd()
lcd.clear()

# init pir sensor
pir_sensor = 5  # pir sensor, random pin assignment
GPIO.setup(pir_sensor, GPIO.IN)

# init lights led
lights_led = 16  # Lights LED, random pin assignment
GPIO.setup(lights_led, GPIO.OUT)
GPIO.output(lights_led, GPIO.LOW)
lights_on = False
last_time_triggered = time.time()
```

For the security system, we will declare the doors and windows pushbutton as GPIO pin 23 and set it up as a pull-up resistor. We will also implement a security_enabled boolean variable to check if the door and windows are closed or not, i.e., if the security is enabled or not. For the temperature systems, we only need to retrieve the local humidity using an API system. To do so, we will create an account on OpenWeatherMap to get our own API key for the Irvine weather. Then, we will send a request for the current weather data in Irvine, use the requests and json modules to send a HTTP GET request to the API and convert the response to JSON format, and access the humidity data by navigating the dictionary returned by the response data. We will use this humidity data throughout our program.

```
# init security system
security_button = 23  # door/window button , random pin
GPIO.setup(security_button, GPIO.IN, pull_up_down=GPIO.PUD_UP)
security_enabled = True

# init api system
url = "http://api.openweathermap.org/data/2.5/weather?q=Irvine,US&appid=bf323aef8f185b426b5e
response = requests.get(url).json()
x = response["main"]
humidity = x["humidity"]
```

For our HVAC system, we will create three boolean variables to keep track of whether the HVAC is currently off or not, and whether the heater or AC is currently off or not. We will declare the DHT11 component as GPIO pin 17, set it up as input, and create a class instance of the DHT module from Freenove. To get the first *accurate* temperature reading, we will need to retrieve the first three readings from the DHT and get the average of that. To do so, we use an empty list called temp_list, and a while loop to append the first three *valid* readings from the DHT component. The for loop in the while loop is taken from the Freenove DHT module to ensure that the DHT reading is valid before actually getting the reading. Afterwards, we will get the average of the temp_list, convert it into Fahrenheit, and multiply it by the humidity from our API, to get the weather index. We will also set the desired temperature to be the same as the weather index for now so that the heater or AC would not be automatically turned on on program startup.

```
# init hvac system
hvac_off = True
heat = False
ac = False
dht_pin = 17  # temp pin assignment
GPIO.setup(dht_pin, GPIO.IN)
dht = DHT.DHT(dht_pin)
temp_list = []
while len(temp_list) < 3:  # append three readings for first temperature
    for i in range(0, 15):
        chk = dht.readDHT11()
        if (chk == 0):
            break
        time.sleep(0.1)
    temp_now = dht.getTemperature()
    temp_list.append(temp_now)
    time.sleep(1)
t = sum(temp_list) / 3
t = int((t * 1.8) + 32)
weather_index = round(t + 0.05 * humidity)  # init weather_index as 0
desired_temp = weather_index  # init desired_temp as 69
```

For the ambient temperature control system, we will use two color coded (blue and red) pushbuttons to increase or decrease the desired temperature. We will assign these pushbuttons to GPIO pins 14 and 18 respectively, and set them as pull-up resistors. The LEDs for the HVAC system (AC and heater) will be declared as GPIO pins 25 and 12 respectively, and set up as output and initially on low. For the fire alarm system, we will use a single boolean variable to indicate whether a fire is detected or not. To implement an internal clock system, we will open a file called “log.txt” to write our event occurrences to.

```
# init buttons for hvac
inc_temp = 14  # temp pin assignment
dec_temp = 18  # temp pin assignment
GPIO.setup(inc_temp, GPIO.IN, pull_up_down=GPIO.PUD_UP)
GPIO.setup(dec_temp, GPIO.IN, pull_up_down=GPIO.PUD_UP)

# init leds for hvac
ac_led = 25  # temp pin assignment
heater_led = 12  # temp pin assignment
GPIO.setup(ac_led, GPIO.OUT)
GPIO.output(ac_led, GPIO.LOW)
GPIO.setup(heater_led, GPIO.OUT)
GPIO.output(heater_led, GPIO.LOW)

# init fire alarm system
fire_alarm = False

# open log file
logfile = open('log.txt', 'w')
```

2.3 Helper Functions

```
# fire alarm flash
def flash_leds():
    GPIO.output(lights_led, GPIO.HIGH)
    GPIO.output(ac_led, GPIO.HIGH)
    GPIO.output(heater_led, GPIO.HIGH)
    time.sleep(0.5)
    GPIO.output(lights_led, GPIO.LOW)
    GPIO.output(ac_led, GPIO.LOW)
    GPIO.output(heater_led, GPIO.LOW)
    time.sleep(0.5)
```

This function will periodically flash the three LEDs on the breadboard over a one second interval if a fire is detected and a fire alarm is triggered. It will be called in the main fire alarm function further in the program.

```
# get ambient temperature
def get_temp():
    for i in range(0, 15):
        chk = dht.readDHT11()
        if (chk == 0):
            break
        time.sleep(0.1)
    temp_now = dht.getTemperature()
    temp_list.append(temp_now)
    if len(temp_list) > 3:
        temp_list.pop(0)
    temp = sum(temp_list) / 3
    return int((temp * 1.8) + 32)  # convert to fahrenheit
```

To periodically update the weather index or weather index, we will create a function that reads a measurement from the DHT11 component every second, append it to the temp_list variable mentioned in the init_everything function while popping the oldest measurement, get the average in that list and return the Fahrenheit value of that average temperature. The for loop code is taken directly from the Freenove DHT module. This loop ensures that the reading we get from the DHT component is *valid* before we actually use it in our calculations.

```
# change desired temperature
def change_desired_temp():
    global desired_temp
    if GPIO.input(inc_temp) == GPIO.LOW and desired_temp < 95:
        desired_temp += 1
        return
    if GPIO.input(dec_temp) == GPIO.LOW and desired_temp > 65:
        desired_temp -= 1
        return
```

This function will read input from the two ambient temperature control pushbuttons and either increment or decrement the desired temperature accordingly, as long as the desired temperature will not increase past 95 or decrease past 65 degrees.

```python
# turn on heater
def turn_on_heater():
    global heat, ac, hvac_off
    heat = True
    log_event('HVAC HEAT')
    ac = False
    GPIO.output(ac_led, GPIO.LOW)  # turn off ac led if on
    hvac_off = False
    lcd.clear()
    lcd.write(5, 0, 'HEATER')  # display message
    lcd.write(7, 1, 'ON')
    GPIO.output(heater_led, GPIO.HIGH)  # turn on heater led
    time.sleep(3)
    lcd.clear()
    return
```

```python
# turn on ac
def turn_on_ac():
    global heat, ac, hvac_off
    heat = False
    GPIO.output(heater_led, GPIO.LOW)  # turn off heater led if on
    ac = True
    log_event('HVAC AC')
    hvac_off = False
    lcd.clear()
    lcd.write(7, 0, 'AC')  # display message
    lcd.write(7, 1, 'ON')
    GPIO.output(ac_led, GPIO.HIGH)  # turn on ac led
    time.sleep(3)
    lcd.clear()
    return
```

These functions will directly turn on the heater and AC. It will first change the respective boolean variables of each function. For example, for the turn_on_heater function, it will set the heat variable to True and set the AC and hvac_off variables to false so that it will not interfere with other parts of the program. Moreover, we will turn off the AC LED if it is on, and log the event occurrence “HVAC HEAT” to the log file using a custom function log_event that will be mentioned later. To display the message that the heater is now turned on, we will clear the LCD and display the message “HEATER ON” for three seconds. The same algorithm is used for the turn_on_ac function.

```python
# turn off hvac
def turn_off_hvac():
    global heat, ac, hvac_off
    heat = False
    ac = False
    hvac_off = True
    log_event('HVAC OFF')
    GPIO.output(heater_led, GPIO.LOW)  # turn off heater led
    GPIO.output(ac_led, GPIO.LOW)  # turn off ac led
    time.sleep(0.3)
    return

# turn on hvac
def turn_on_hvac():
    # turn on heat or ac
    if weather_index <= desired_temp - 3:
        turn_on_heater()
    else:
        if weather_index >= desired_temp + 3:
            turn_on_ac()
```

To turn off the HVAC, we will create a function turn_off_hvac that sets the heat and AC boolean variables to False, and the hvac_off variable to True. We will also log the event occurrence “HVAC OFF” and turn off the heater and AC LEDs if it is on. No message is displayed on the LCD because it is not required to. We will only call this function if the security system is disabled and the door/window is open, in which the HVAC must be halted. If the security is enabled and the door/window is closed, we need to create a function that turns on the HVAC. To do so, turn_on_hvac will use an if-else statement to call either turn_on_heater or turn_on_ac

mentioned earlier, if the desired temperature is either 3 degrees below or above the weather index.

```
# save logs to file
def log_event(event):
    global logfile
    timestamp = datetime.now().strftime('%H:%M:%S')
    logfile.write(f"{timestamp} {event}\n")
    logfile.flush()
```

This function will log every event occurrence in the BMS system by using a string argument. It will get the current system's timestamp using the datetime module and write it to the logfile declared in the init_everything function at the start of the program.

2.4 Run Systems Functions

```
# run PIR sensor + ambient light control system
def run_pir_sensor():
    global lights_on, last_time_triggered, pir_sensor, lights_led
    # motion detected
    if GPIO.input(pir_sensor):
        log_event('MOTION DETECTED')
        timestamp = datetime.now().strftime('%H:%M:%S')
        print("motion detected at: ", timestamp)
        # if lights not on, turn it on
        if not lights_on:
            GPIO.output(lights_led, GPIO.HIGH)  # turn on lights led
            lights_on = True
            log_event('LIGHTS ON')
        last_time_triggered = time.time()
    # no motion detected
    else:
        # if lights on, check if it has been > 10 secs
        if lights_on and (time.time() - last_time_triggered >= 10):
            GPIO.output(lights_led, GPIO.LOW)  # turn off lights led
            lights_on = False
            log_event('LIGHTS OFF')
        # if lights off or has not been > 10 secs, continue
    # time.sleep(0.3)
```

This function will run the PIR sensor as well as the ambient lighting control system. It will use the lights_on, last_time_triggered, pir_sensor, and lights_led variables declared in the init_everything function. It first checks if motion is detected by checking if there is input on the PIR sensor pin. If motion is detected, we will log the event occurrence, turn on the lights LED if it is not already on, log the event occurrence that the lights are turned on, and update the last_time_triggered variable to the current time to reset the automatic turn off timer. If no motion

is detected, we will check if we need to turn off the lights LED or not. To do so, we will use an if statement to check if the lights LED is on and whether 10 seconds has elapsed since the last time the sensor was triggered. If so, we will turn off the lights LED, set the lights_on boolean variable as False, and log the event occurrence that the lights are turned off. If the lights are already off or have not been more than 10 seconds, we will continue.

```
# run fire alarm system
def run_fire_alarm():
    global fire_alarm, security_enabled, weather_inde
    fire_alarm = True
    log_event('FIRE ALARM ON')

    # open window/door
    lcd.clear()
    security_enabled = False
    log_event('DOOR OPEN')
    log_event('HVAC OFF')
    lcd.write(1, 0, 'Window/Door  O')
    lcd.write(2, 1, 'HVAC  HALTED')
    turn_off_hvac()
    time.sleep(3)  # display for 3 seconds

    # display emergency message
    lcd.clear()
    lcd.write(3, 0, 'FIRE! Dr: O')
    lcd.write(0, 1, 'Please evacuate!')
```

```
# flashes leds
while fire_alarm:
    flash_leds()
    ct = get_temp()
    weather_index = round(ct + 0.05 * humidity)
    if weather_index < 95:
        fire_alarm = False
        lcd.clear()
        lcd.write(0, 0, 'Resuming normal')
        lcd.write(1, 1, 'operations...')
        time.sleep(3)
# resume normal operations
log_event('FIRE ALARM OFF')
lcd.clear()
return
```

If a fire is detected, the fire alarm system will be triggered via the run_fire_alarm function. This function will use the fire_alarm, security_enabled, and weather_index variables declared in the init_everything function. It will first set the fire_alarm boolean variable to True and log the event occurrence “FIRE ALARM ON”. Then, it will open the door/window by disabling the security, setting the security_enabled to False, log the event occurrences and display the messages that the door is open and HVAC is turned off for 3 seconds. Afterwards, we will display an emergency message indicating that there is a fire and that everyone must evacuate through the opened door. We will then flash the LEDs using a while loop that checks that the fire_alarm variable is still True or not. For every iteration of the while loop, we will call the flash_leds function and get the ambient temperature/weather index to check if the fire is resolved or not. If the ambient temperature/weather index has decreased to below 95 degrees, then we can set the fire_alarm to False to exit the while loop in the next iteration, and display the message that we are now resuming normal operations. We will log the event that the fire alarm is now off before exiting the function.

```python
# run security system
def run_security():
    global security_enabled, security_button, lcd
    # button pressed
    if GPIO.input(security_button) == GPIO.LOW:
        # open/close door/window, turn on/off HVAC
        security_enabled = not security_enabled
        lcd.clear()

        # if door/window open, turn off HVAC
        if not security_enabled:
            log_event('DOOR OPEN')
            log_event('HVAC OFF')
            lcd.write(1, 0, 'Window/Door  O')
            lcd.write(2, 1, 'HVAC  HALTED')
            time.sleep(3)  # display for 3 seconds
            turn_off_hvac()
            lcd.clear()
            return
        else:
            log_event('DOOR CLOSED')
            lcd.write(1, 0, 'Window/Door  C')
            lcd.write(4, 1, 'HVAC  ON')
            time.sleep(3)  # display for 3 seconds
            turn_on_hvac()
            lcd.clear()
            return
```

For the security system, we will create a function that uses the security_enabled, security_button, and lcd variables declared in the init_everything function. This run_security function will detect whether the security pushbutton is pressed or not. If so, we will invert the truth value of the security_enabled variable and take the necessary actions based on security_enabled. If the security is disabled, we will log the event occurrences that the door is opened and the HVAC is turned off. We will also call turn_off_hvac and display the messages that the door/window is opened and the HVAC is now halted for 3 seconds. If the security is enabled, we will log the event occurrences that the door is closed and the HVAC is turned on. We will also call turn_on_hvac and display the messages that the door/window is closed and the HVAC is now on for 3 seconds.

```python
# run temperature control system
def run_hvac():
    global heat, ac, weather_index, desired_temp, security_enabled
    # listen for desired temp change and turn on/off hvac accordingly
    change_desired_temp()
    # time.sleep(0.5)
    if (weather_index <= desired_temp - 3) and not heat and security_enabled:
        turn_on_heater()
    else:
        if (weather_index >= desired_temp + 3) and not ac and security_enabled:
            turn_on_ac()
```

For the ambient temperature control system, we will create a function that uses the heat, ac, weather_index, desired_temp, and security_enabled variables declared in the init_everything function. This run_hvac function will call the change_desired_temp function to listen for any input from the temperature change pushbuttons and turn on the heater or AC depending on the conditions. If the desired temperature is 3 degrees higher than the weather index and the heater is not already on, and the security is enabled (door/window are closed), then we will call the turn_on_heater function to turn on the heater and its LED. Conversely, if the desired temperature is 3 degrees lower than the weather index and the AC is not already on, and the security is enabled (door/window are closed), then we will call the turn_on_ac function to turn on the AC and its LED.

2.5 Main Functions

```
# display all statuses
def display_statuses():
    global weather_index, humidity, desired_temp
    # lights status
    if lights_on:
        lcd.write(12, 1, 'L:ON')
    else:
        lcd.write(11, 1, 'L:OFF')
    # window/door status
    if not security_enabled:
        lcd.write(12, 0, 'Dr:O')
    else:
        lcd.write(12, 0, 'Dr:C')
```

```
# hvac status
if hvac_off:
    lcd.write(0, 1, 'H:OFF')
elif heat:
    lcd.write(0, 1, 'H:HEAT')
elif ac:
    lcd.write(0, 1, 'H:AC')
else:
    lcd.write(0, 1, 'H:ERR')  #fallback

# update temps
current_temp = get_temp()
weather_index = round(current_temp + 0.05 * humidity)
lcd.write(0, 0, str(desired_temp))
lcd.write(2, 0, "/" + str(weather_index))

#update time
timestamp = datetime.now().strftime('%H:%M')
lcd.write(7, 0, timestamp)
```

This function is one of the core functions that will display all of the statuses onto the LCD and will be updated live. It will display the status of the lights, door, HVAC, temperatures, and time based on the boolean variables declared in the init_everything function at the start of the program. These boolean variables are modified using the run functions and helper functions throughout the program and will determine what to output onto the LCD. Everything is somewhat self explanatory. For the temperature, we will calculate the weather index (using the same algorithm mentioned above) and output it and the desired temperature every time display_statuses is called. For the time, we will just display the hour and minute in 24-hour format on the top row of the LCD because space is permitted in that row only.

```python
def main():
    init_everything()
    global weather_index, humidity, desired_temp, temp_list
    while True:
        lcd.clear()
        if weather_index >= 95:
            run_fire_alarm()
        run_pir_sensor()
        run_security()
        run_hvac()
        display_statuses()
        time.sleep(0.5)

try:
    main()
finally:
    global logfile
    logfile.close()
    lcd.clear()
    GPIO.cleanup()
```

My main function will run every function mentioned throughout the program. It will first call init_everything to initialize all of the sensors, actuators, LEDs, buttons, and its pin assignments and input/output declarations. Then, in an infinite loop, we will clear the LCD to update the statuses live. For every iteration, we will check if the ambient temperature/weather index is above 95 degrees or not. If it is, we will call run_fire_alarm which will halt the remaining code and flash the LEDs in the while loop mentioned before. If there is no fire case, we will run the PIR sensor, security system, and HVAC system using the run functions mentioned above. We will display everything onto the LCD using the display_statuses function. I opted to set the iteration timer to 0.5 seconds for smoother interfacing, visuals, and functionalities.

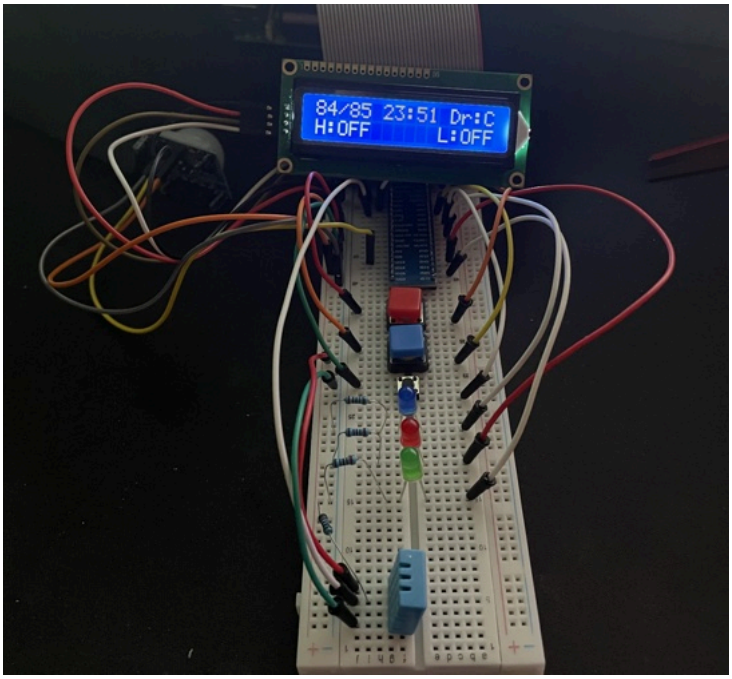
2.6 Bonus Points

```python
#update time
timestamp = datetime.now().strftime('%H:%M')
lcd.write(7, 0, timestamp)
```

For the 10% bonus points, I incorporated the actual time in 24-hour format on the top row of the LCD. I used the datetime module to get the current system's time and output only the hour and minute because there was not enough space for the seconds.

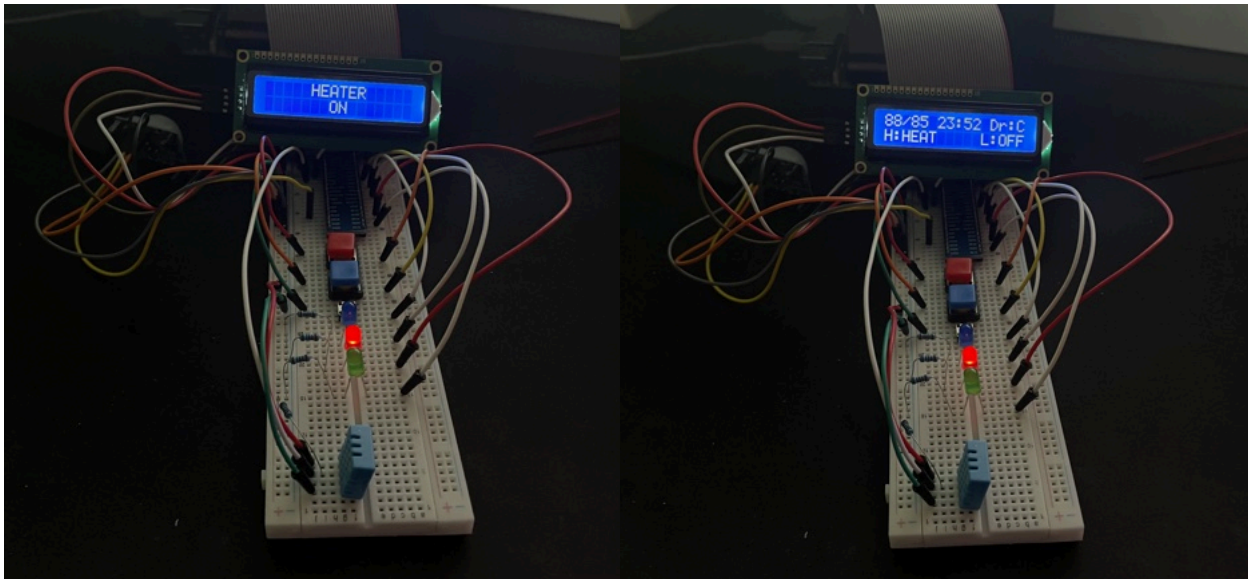
3 Simulation Results

3.1 Initial Startup



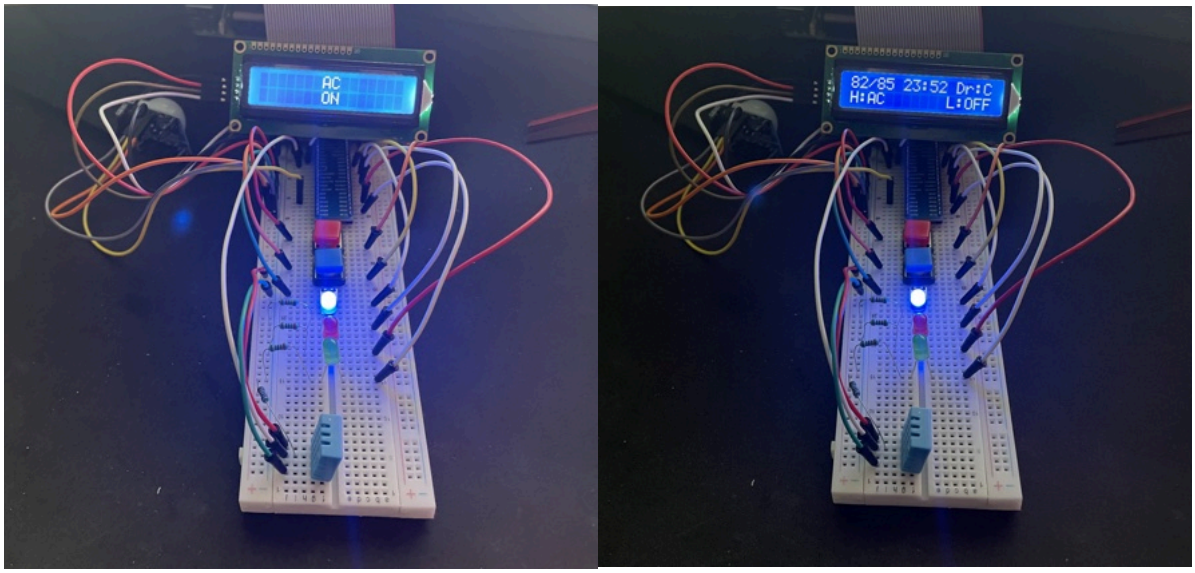
On startup, the desired temperature is the same as the ambient temperature/weather index but the ambient temperature/weather index increased by one by the time I took a picture. Otherwise, the time is displayed in 24-hour format on the top row, the door is initially closed, the HVAC is off, and the lights are off.

3.2 Heater Functionality



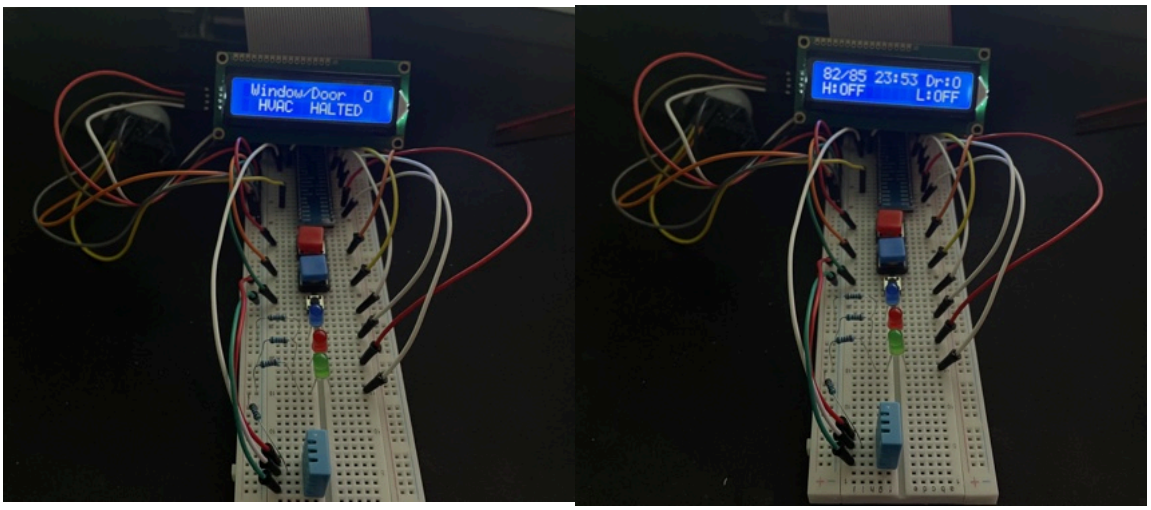
For the heater, I increased the desired temperature to 88 to turn on the heater since the desired temperature is now 3 degrees or more above the ambient temperature/weather index. Once this is recognized, the red heater LED will be turned on, a message is displayed on the LCD to indicate that the heater is turned on, and after 3 seconds, it will be dismissed and the HVAC status will show that the heater is now turned on: “H: HEAT”.

3.3 AC Functionality



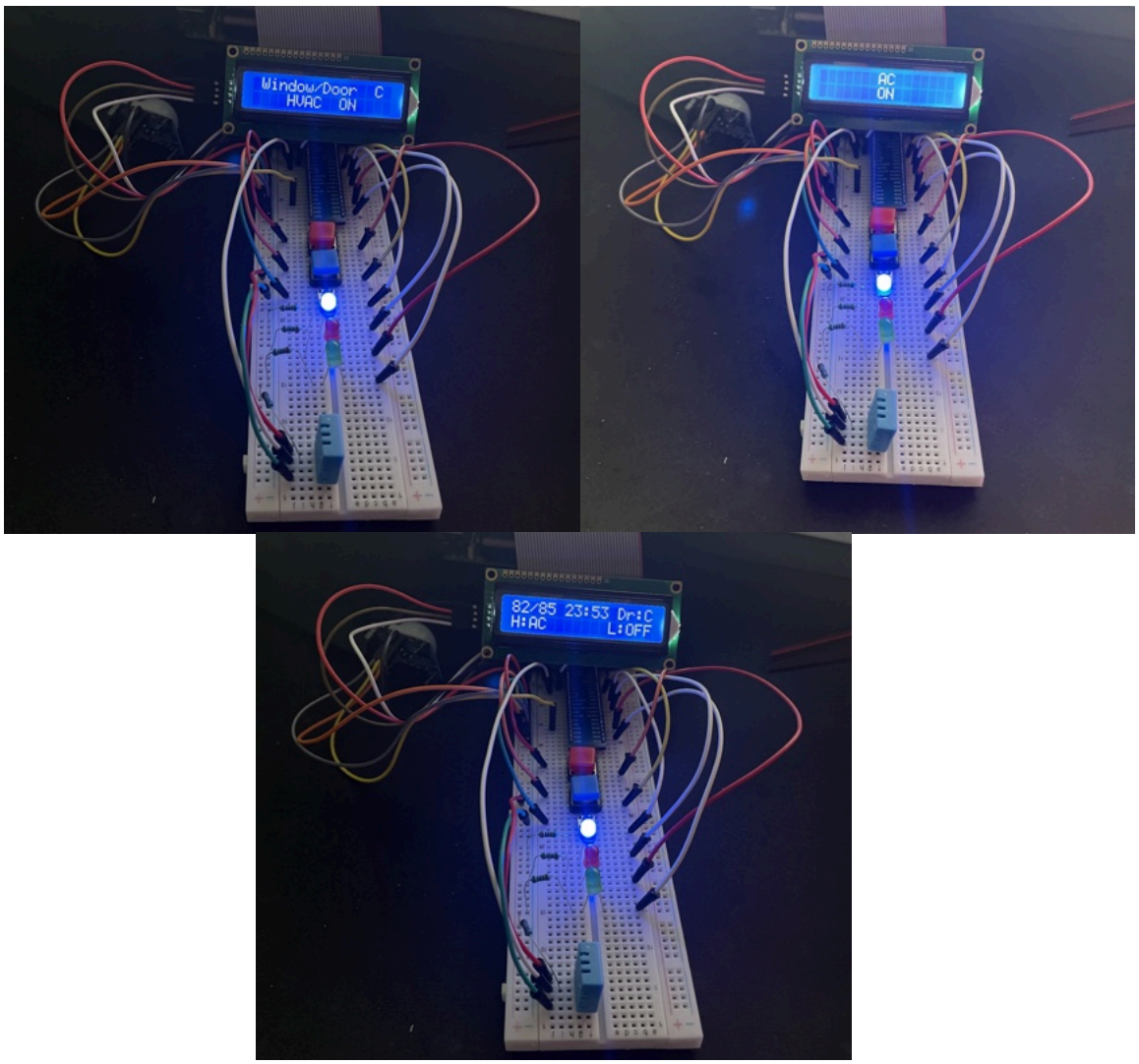
For the AC, I decreased the desired temperature to 82 to turn on the AC since the desired temperature is now 3 degrees or more below the ambient temperature/weather index. Once this is recognized, the blue AC LED will be turned on, a message is displayed on the LCD to indicate that the AC is turned on, and after 3 seconds, it will be dismissed and the HVAC status will show that the heater is now turned on: “H: AC”.

3.4 Security Disabled



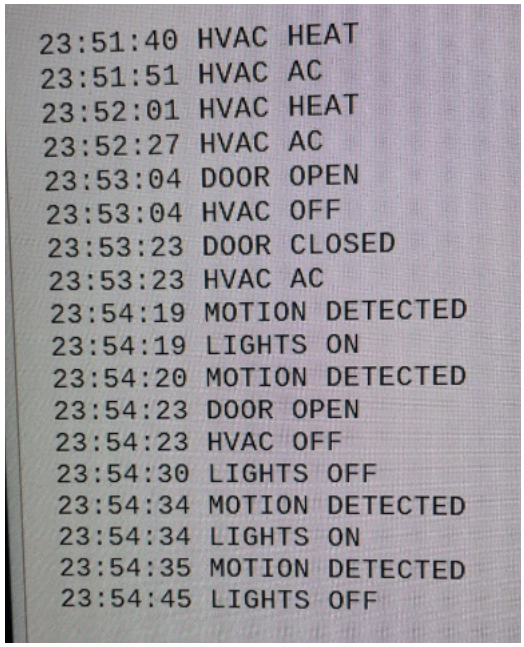
For the security system, if we disable it, a message will be displayed for 3 seconds indicating that the window/door is now opened, and the HVAC is halted. The blue AC LED from before is now turned off and the HVAC status is now set to “OFF”.

3.5 Security Enabled



For the security system, if we enable it, a message will be displayed for 3 seconds indicating that the window/door is now closed, and the HVAC is turned back on. The blue AC LED from before is now turned on since the desired temperature is 3 degrees or more below the ambient temperature/weather index. Before returning to the main status screen, a message displaying that the AC is now on is displayed, and the HVAC status is now set to “AC”.

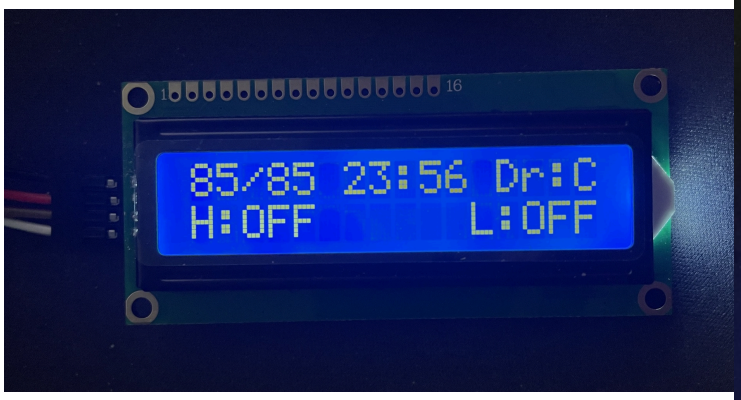
3.6 Log File

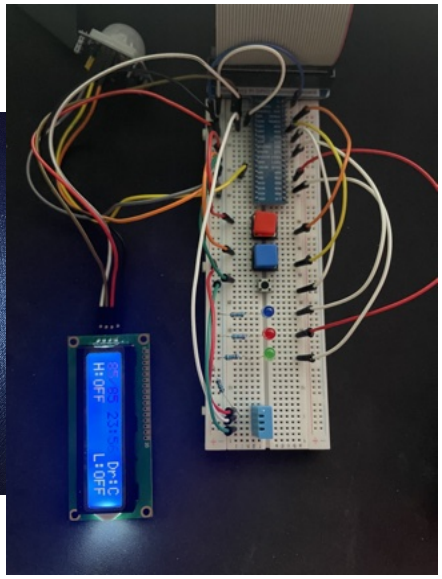


```
23:51:40 HVAC HEAT
23:51:51 HVAC AC
23:52:01 HVAC HEAT
23:52:27 HVAC AC
23:53:04 DOOR OPEN
23:53:04 HVAC OFF
23:53:23 DOOR CLOSED
23:53:23 HVAC AC
23:54:19 MOTION DETECTED
23:54:19 LIGHTS ON
23:54:20 MOTION DETECTED
23:54:23 DOOR OPEN
23:54:23 HVAC OFF
23:54:30 LIGHTS OFF
23:54:34 MOTION DETECTED
23:54:34 LIGHTS ON
23:54:35 MOTION DETECTED
23:54:45 LIGHTS OFF
```

The log file is displayed on the system terminal as well as the same directory the program is run in. The following timestamps are aligned with the time every event occurrence is shown in the simulation results and can be verified by checking the time displayed on the previous pictures.

3.7 LCD and Wiring





This is the general LCD display output and an overview of the wiring of my BMS system.

Thank you! Have a great summer!